Deep Software Stack Optimization for AI-Enabled Embedded Systems

Sep. 28, 2025

Prof. Seongsoo Hong, Namcheol Lee, Geonha Park, and Taehyun Kim
{sshong, nclee, ghpark, thkim}@redwood.snu.ac.kr
Dept. of Electrical and Computer Engineering,
Seoul National University

Special Thanks to

- Sapphire Stream Technology
  - For sponsoring this tutorial and providing RUBIK Pi 3 boards to attendees

https://www.sapphirestreamtech.com/

Tutorial Outline

Lecture 1: *Exercise Overview and Setup* (1 h 30 min)	
Lecturer	Seongsoo Hong
Topics	Motivating Example
	Development Environment Setup
Coffee Break (30 min)	
Lecture 2: *From Inference Driver to Inference Runtime* (1 h 30 min)	
Lecturer	Seongsoo Hong and Namcheol Lee
Topics	Step-by-Step Inference Driver Walkthrough
	Internals of LiteRT
Lunch Break (1 h)	
Lecture 3: *Model Slicer* (1 h 30 min)	
Lecturer	Seongsoo Hong
Topic	Model Slicer: Slicing and Conversion Tool for LiteRT
Coffee Break (30 min)	
Lecture 4: *Throughput Enhancement on Heterogeneous Accelerators* (1 h 30 min)	
Lecturer	Namcheol Lee
Topic	Implementing a Pipelined Inference Driver for Heterogeneous Processors

Contents

I. **Our Exercise**
II. Tutorial Execution Environment
III. Hands-On Exercise: Connect to RUBIK Pi

On-Device AI

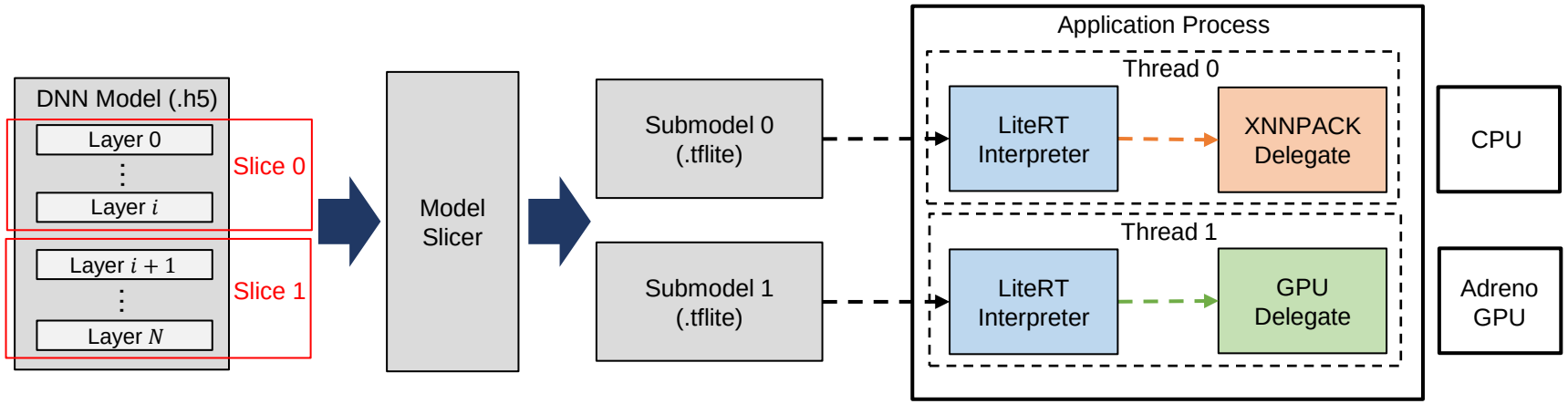
- Refers to "*executing AI models directly on edge devices*"
  - Benefits
    - Enables *real-time*, *privacy-preserving*, and *offline inference*
  - Challenge
    - Operates under limited resources available on edge devices
  - Solutions
    - Static model optimizations
      - Quantization, pruning, knowledge distillation, low-rank adaptation, weight streaming, and sparse inference
    - Dynamic optimizations
      - Pipelined inference, operator selection/scheduling, and operator fusion via capabilities
    - Dynamic resource management
      - Accelerator allocation, memory management, QoS regulation, etc.

Our Exercise Problem

- Lossless stream processing in an edge device
  - Real-world relevance
    - Surveillance cameras
    - Automotive perception applications
  - A transient surge in input data stream may result in data loss
    - When input stream's data rate exceeds device inference throughput
  - Need to scale up inference throughput dynamically

DNN Pipelining (1)

- Scale up inference throughput via pipelining on multiple accelerators
  1. Partition DNN model into submodels
  2. Create worker threads that perform inference for submodels
  3. Allocate the worker threads to available, heterogeneous accelerators



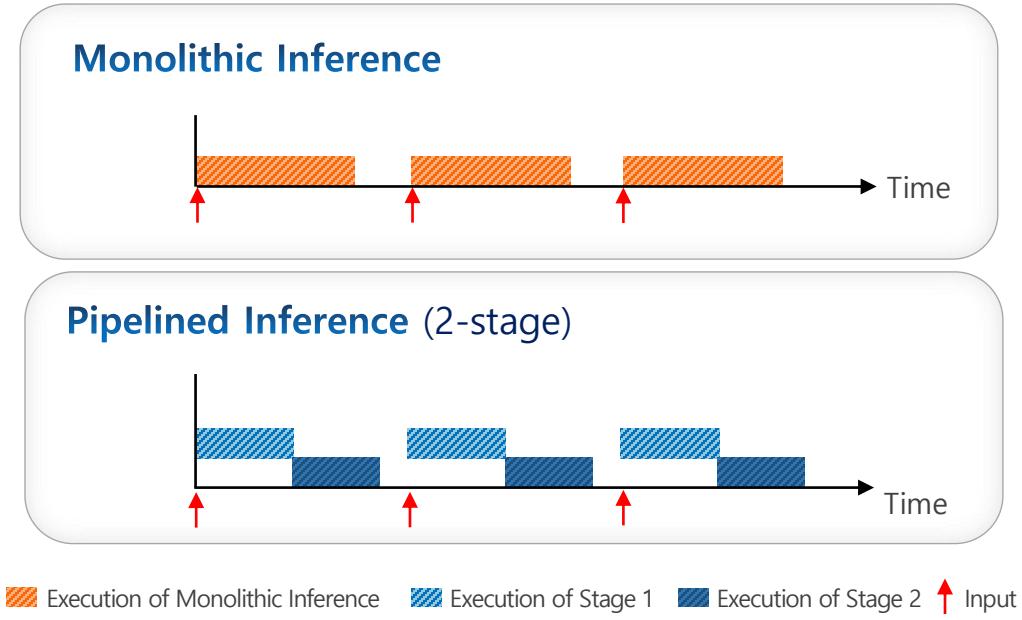
DNN Pipelining (2)

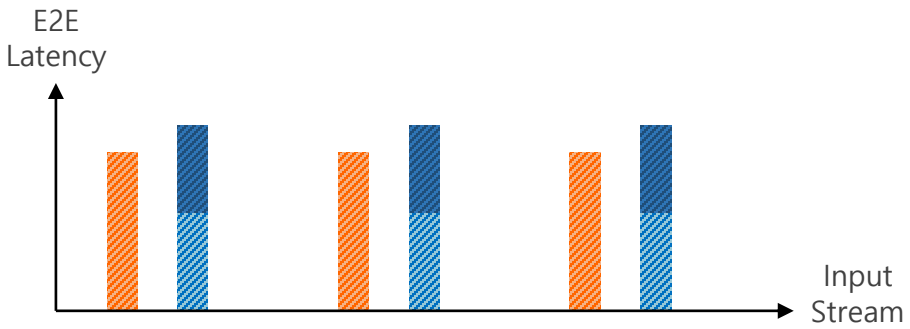
- What is the problem? And why is pipelining beneficial?

- Case analysis
  1. Normal case: *If input stream's data rate ≤ device's inference throughput*
     - *Pipelining can be detrimental*
       - May increase end-to-end (E2E) latency for DNN inference
         - Some stages may run on relatively slower processor (e.g., CPU)
         - Communication delays could be added
  2. Exceptional case: *If input stream's data rate > device's inference throughput*
     - *Pipelining is beneficial*
       - The legacy system may incur unbounded E2E latency due to back logs
       - Pipelining can increase throughput, enabling bounded E2E latency

DNN Pipelining (3)

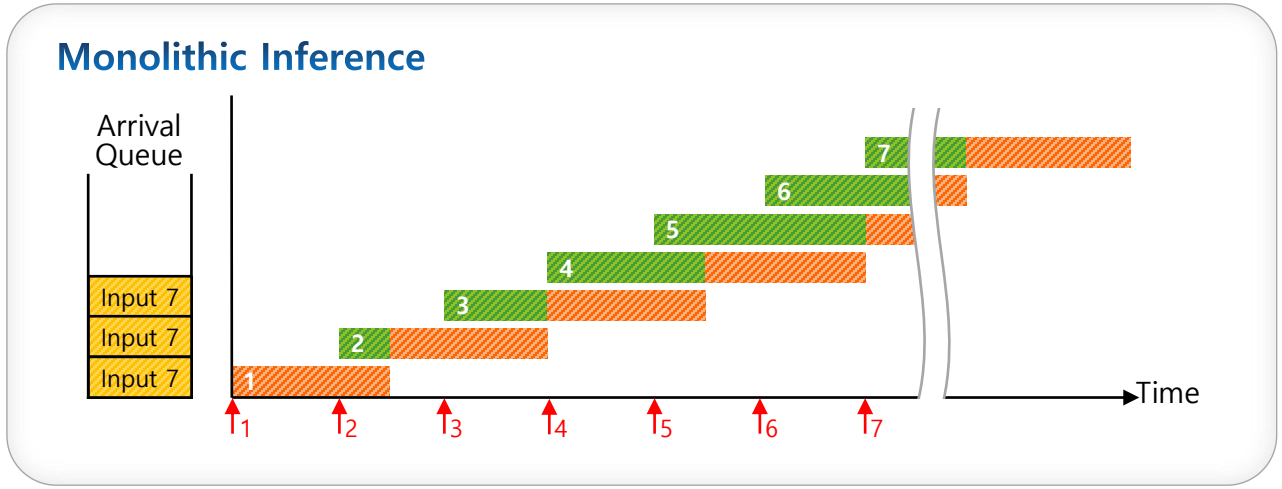
1. Normal case: *If input stream's data rate ≤ device's inference throughput*

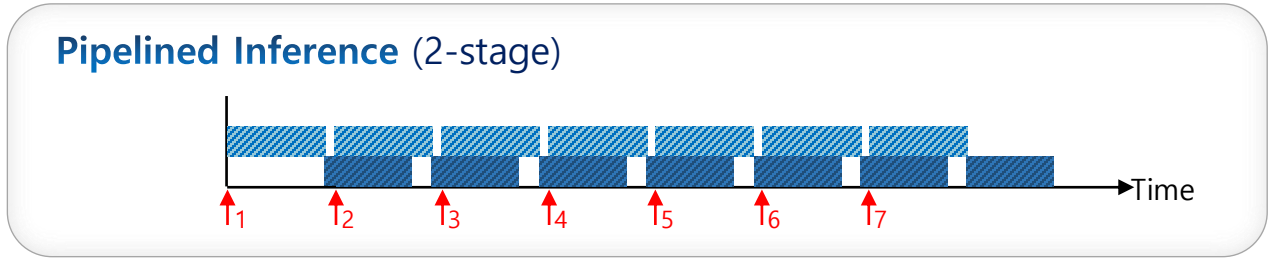


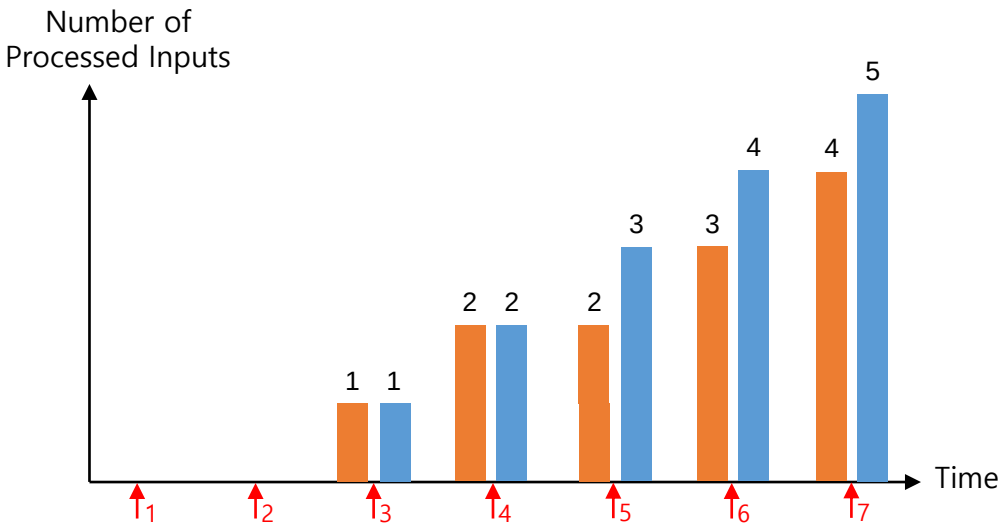


DNN Pipelining (4)

2. *If input stream's data rate > device's inference throughput*



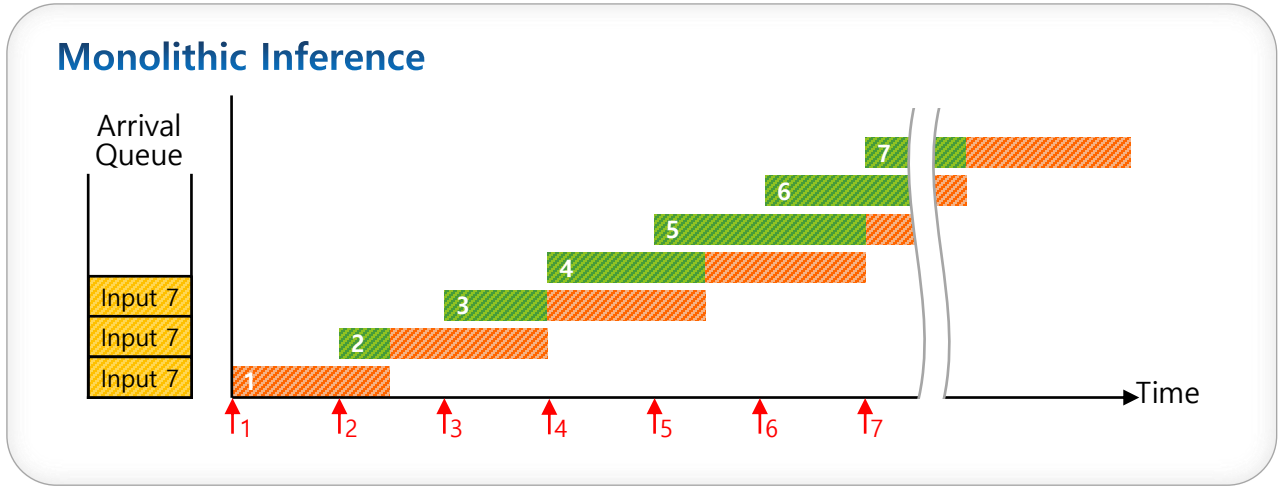


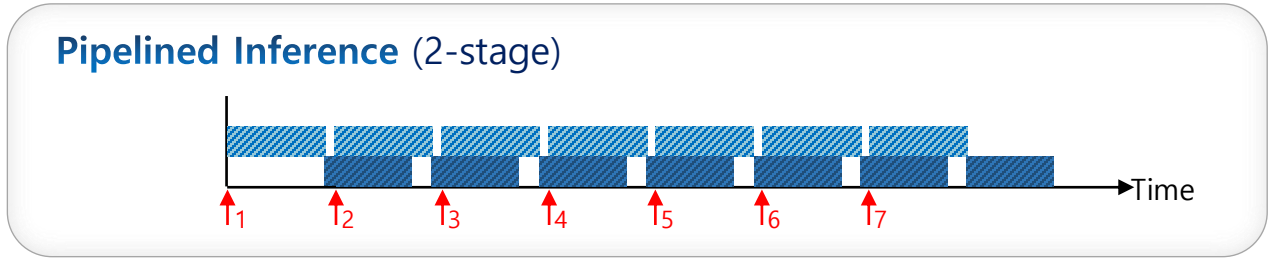


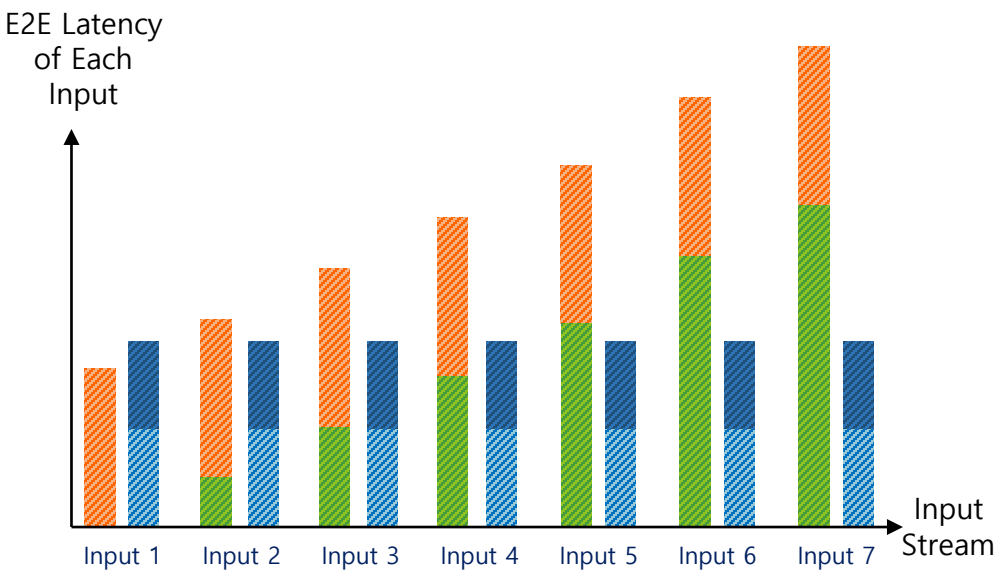
Execution of Monolithic Inference
Queuing Delay
Execution of Stage 1
Execution of Stage 2
$\uparrow_i$ Input i

DNN Pipelining (5)

2. *If input stream's data rate > device's inference throughput*







Execution of Monolithic Inference
Queuing Delay
Execution of Stage 1
Execution of Stage 2
Input i

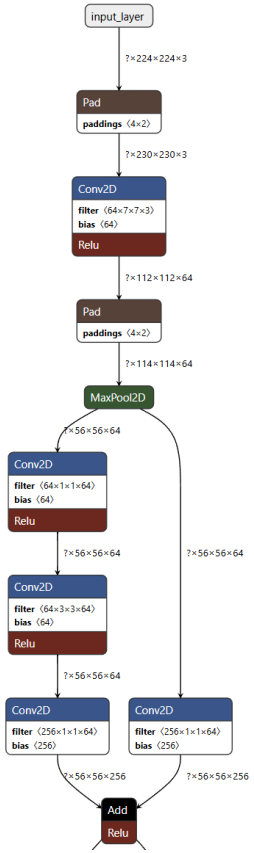
Target DNN Model: ResNet50 (1)

- We will use ResNet50 FP32 throughout the tutorial which is
  - A 50-layer deep convolutional neural network for image classification
  - Introduced in 2015 by He et al.[1] with the concept of skip connections
- Why ResNet50?
  - Commonly used for frame-by-frame classification
  - Moderate computational demand
  - Widely used as a benchmark model in inference optimization

[1]He, Kaiming, et al. "Deep residual learning for image recognition." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016.

Target DNN Model: ResNet50 (2)

- Key details
  - Input
    - 224x224 RGB image (224x224x3)
  - Output
    - 1x1000 tensor
- Observe model structure using Netron[2]
  - **`netron ./models/resnet50.tflite`**



[2]https://netron.app/

Contents

I. Our Exercise

II. **Tutorial Execution Environment**

III. Hands-On : Connect to RUBIK Pi

Hardware: RUBIK Pi 3 (1)

- Overview
  - A Singe Board Computer (SBC)
    built on Qualcomm AI platforms for developers
    - Based on QCS6490 SoC
  - Supports multiple operating systems such as
    - Android 13, Qualcomm Linux, Debian 13
  - Supports SDKs with Qualcomm AI stack
  - Designed and manufactured by Thundercomm

Hardware: RUBIK Pi 3 (2)

- Specification
  - RUBIK Pi 3 with QCS6490 SoC
    - CPU: 8 x Kryo 670 cores
    - GPU: Adreno 643L
    - NPU: Hexagon 770
    - RAM: 8 GB LPDDR4X
    - Storage: 128 GB UFS 2.2

Hardware: RUBIK Pi 3 (3)

- Comparison with Raspberry Pi 5
  - RUBIK Pi offers superior AI performance and faster storage, making it ideal for edge AI education and prototyping

	RUBIK Pi 3	Raspberry Pi 5
CPU	**8 x Kryo 670** (1 x Cortex-A78 @ 2.7 GHz, 3 x Cortex-A78 @ 2.4 GHz, 4 x Cortex-A55 @ 1.9 GHz)	4 x Cortex-A76 @ 2.4GHz
GPU	**Adreno 643L @ Up to 812 MHz**	VideoCore VII @ Up to 800MHz
NPU	**Hexagon 770 (12 TOPS)**	N/A
RAM	8GB	Up to 16GB
ROM	**128GB UFS 2.2**	N/A (Need to purchase external SD card separately)
Price	$179	$120 (16 GB Model)

Hardware: RUBIK Pi 3 (4)

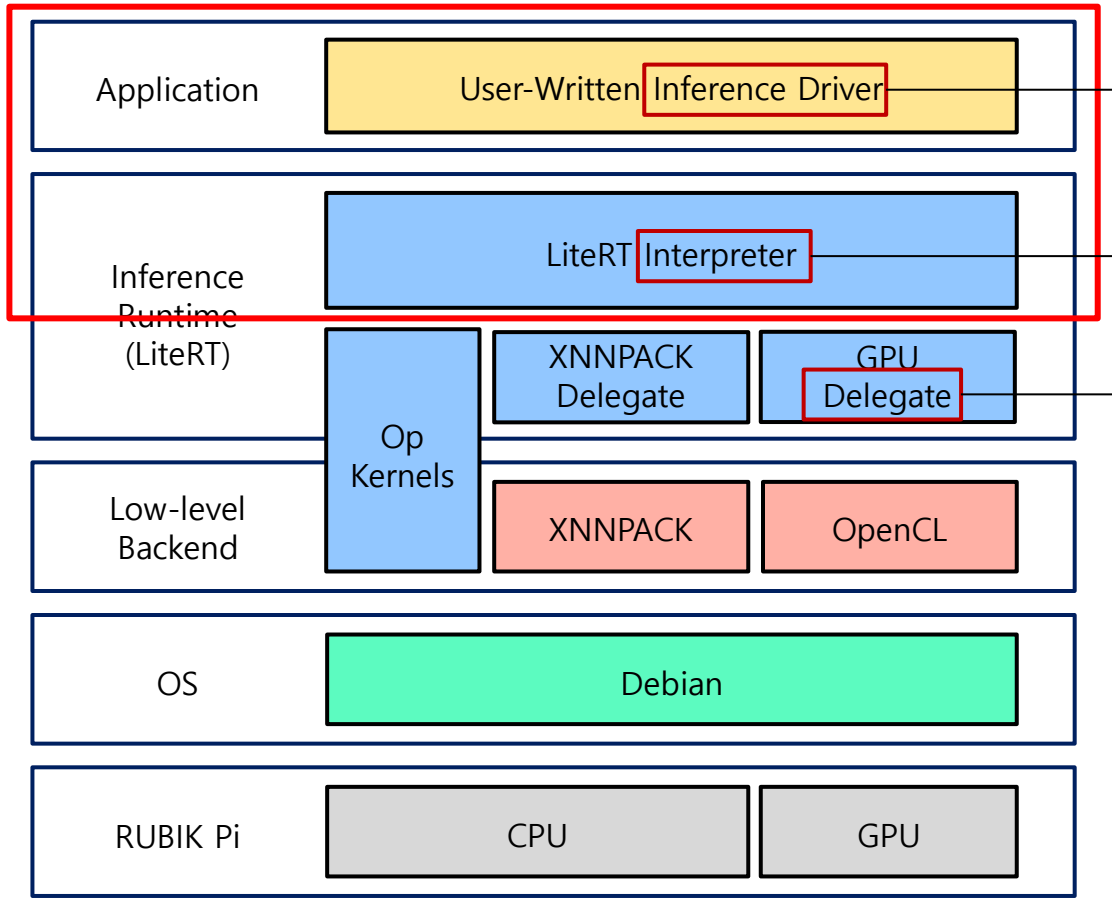
- Ideal for Hands-On learning of embedded AI software stack
  - RUBIK Pi 3 with QCS6490 SoC enables realistic, practice-oriented exercises aligned with real-world industrial applications

Source: Thundercomm (https://www.thundercomm.com/rubik-pi-3-embedded-world-2025/)

Taxonomy

- Inference driver
  - Program that calls the inference runtime's APIs to drive inference
- Inference runtime
  - A software environment that executes DNN inference
    - Such as LiteRT (Google), QNN (Qualcomm), TensorRT (Nvidia)
- LiteRT interpreter
  - A component that prepares and runs inference
- Delegate
  - A component that offloads parts of a model to optimized implementations for faster or more efficient execution

Software Stack on RUBIK Pi 3



- Program that calls the inference runtime's APIs to drive inference
- A component that prepares and runs inference
- A component that offloads parts of a model to optimized implementations for faster or more efficient execution

Contents

I. Our Exercise
II. Tutorial Execution Environment
III. **Hands-On Exercise: Connect to RUBIK Pi**

Connect to RUBIK Pi

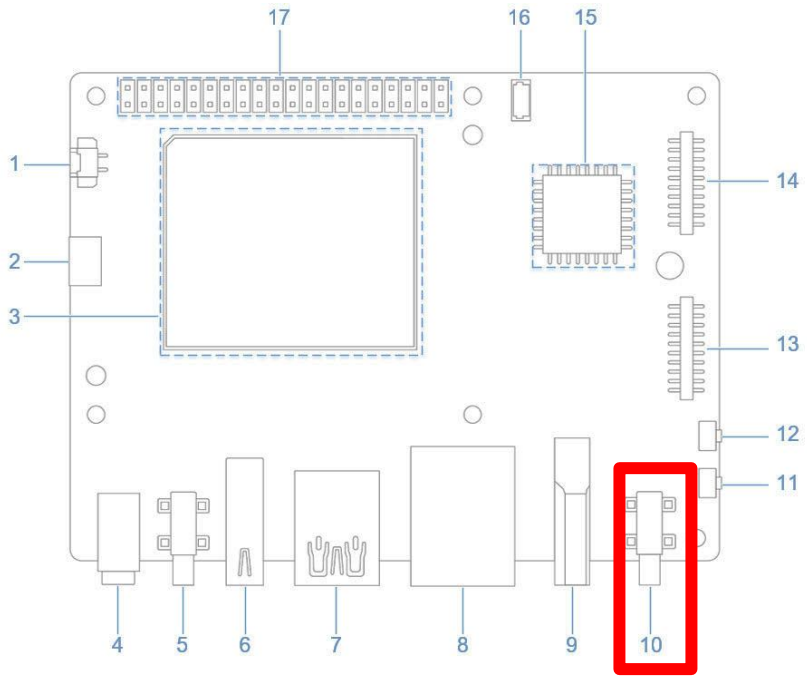
- Objective
  - Establish an SSH connection from your laptop to RUBIK Pi
- Do
  - Follow the instructions in this section
- Verify
  - Your VS Code should show a SSH session connected to RUBIK Pi
- Time
  - 60 minutes

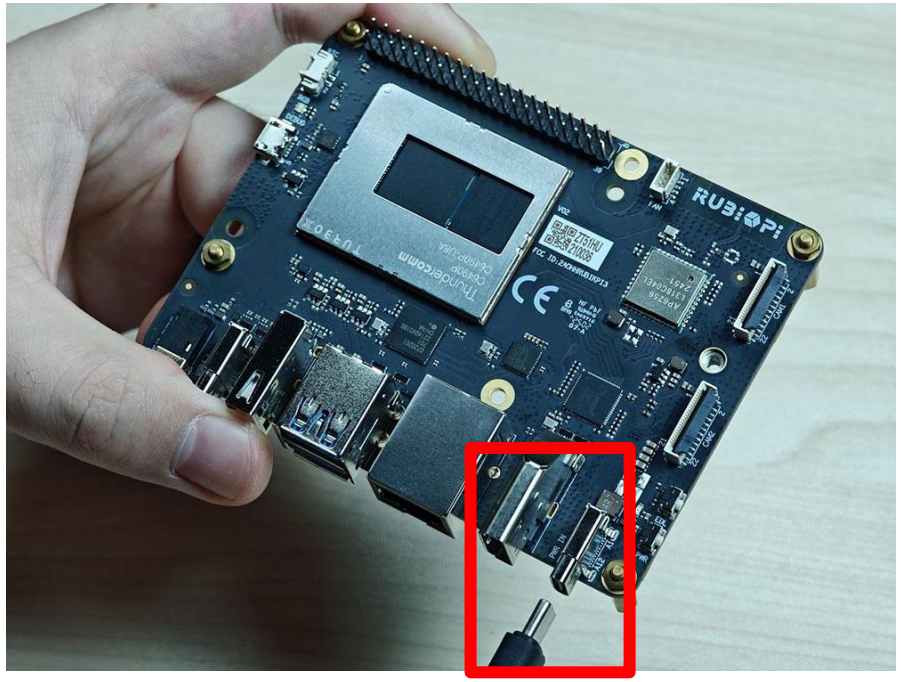
Supply Power (1)

- Connect power cable to the external power supply, and make sure it is switched on

Supply Power (2)

- Connect Type-C power cable to port 10 on RUBIK Pi



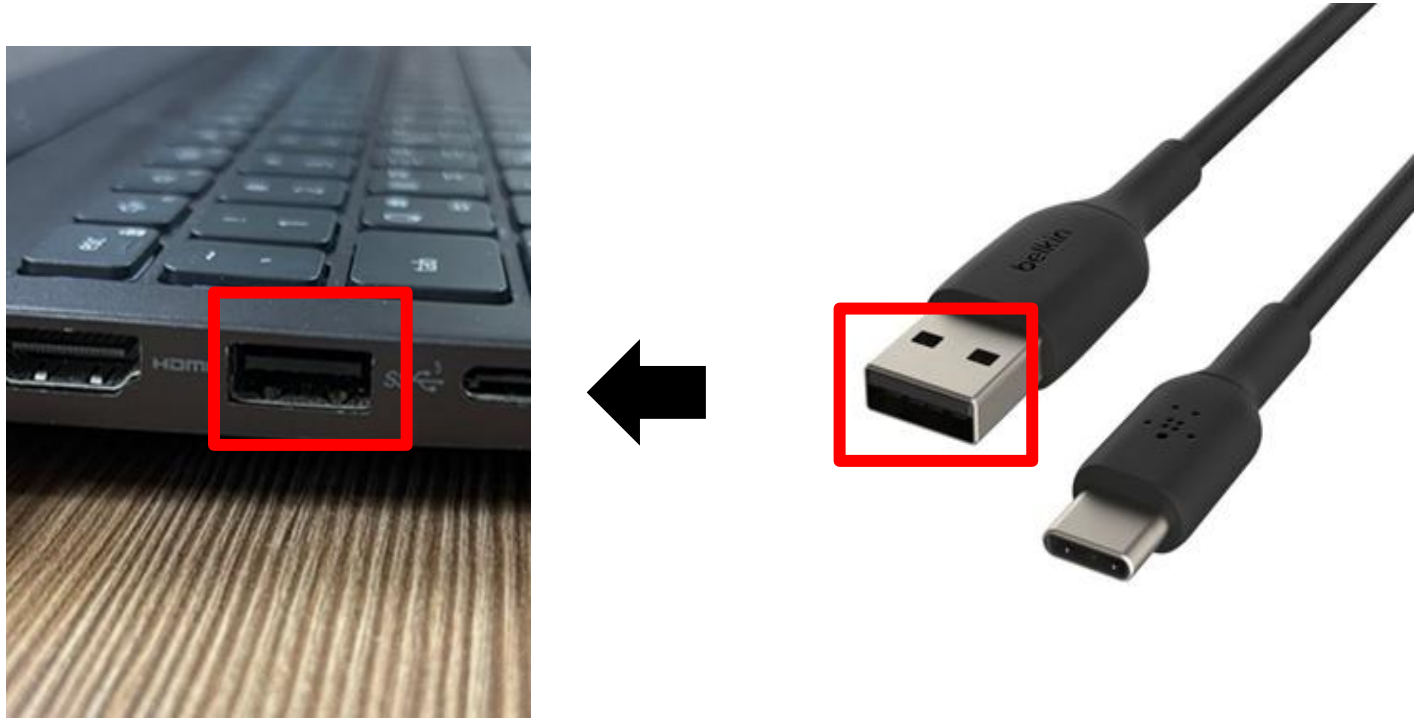


Connect RUBIK Pi to Host PC

- We will access the RUBIK Pi using SSH with Visual Studio Code for development ease
  - To do so
    1. Connect the device to the host PC with a USB-A to USB-C cable
    2. Establish the device's Wi-Fi connection using ADB (Android Debug Bridge)
    3. Make an SSH connection to the device

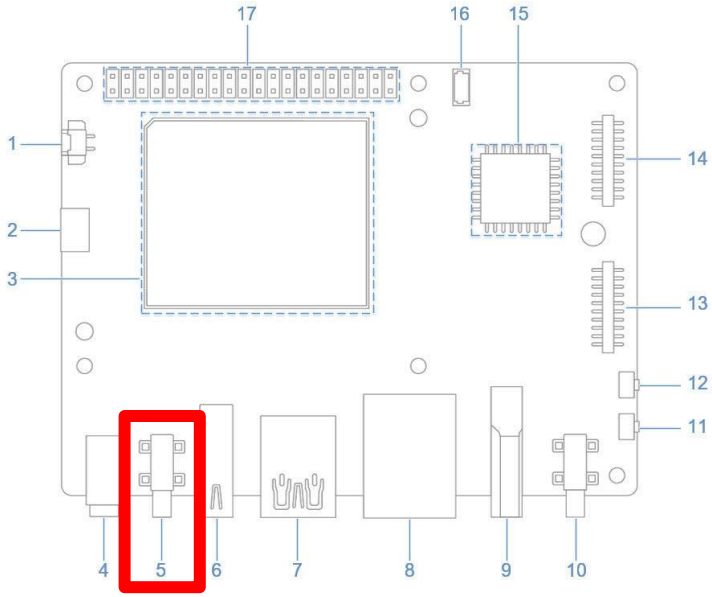
1. Connect Device to Host PC with USB-A to USB-C Cable (1)

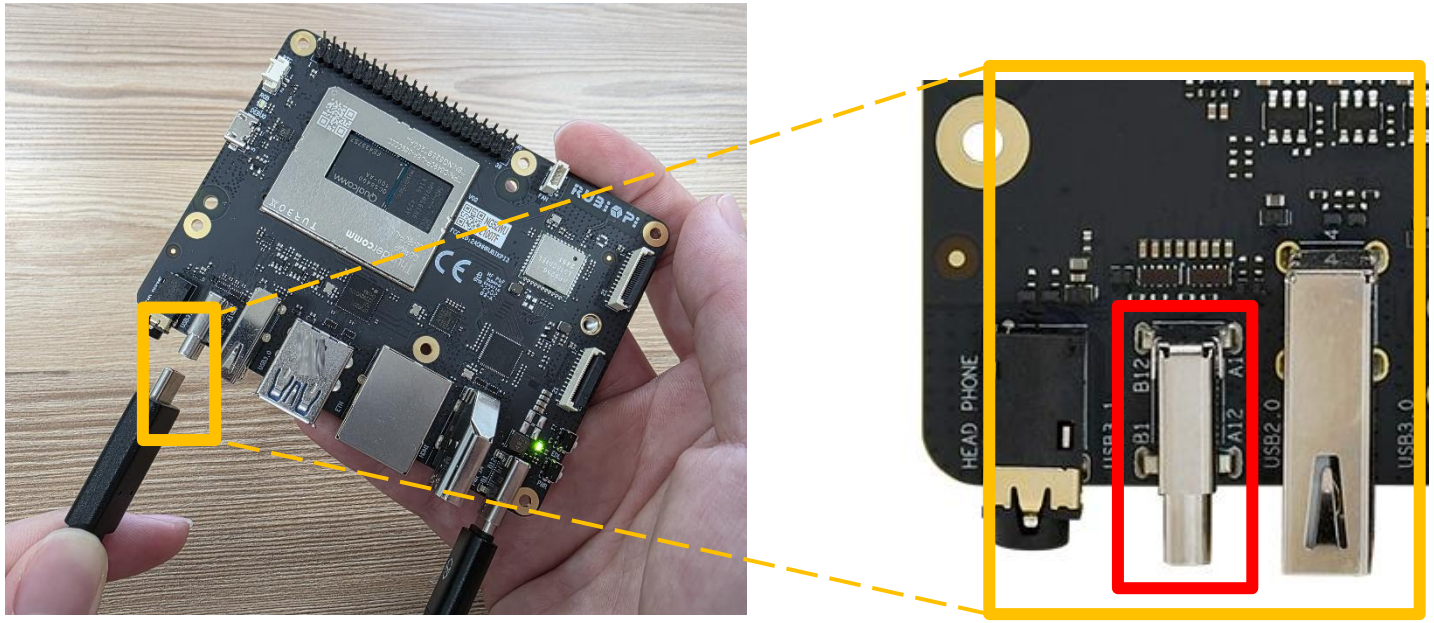
- Connect the USB-A end of the cable to host PC



1. Connect Device to Host PC with USB-A to USB-C Cable (2)

- Connect the USB-C end of the cable to port 5





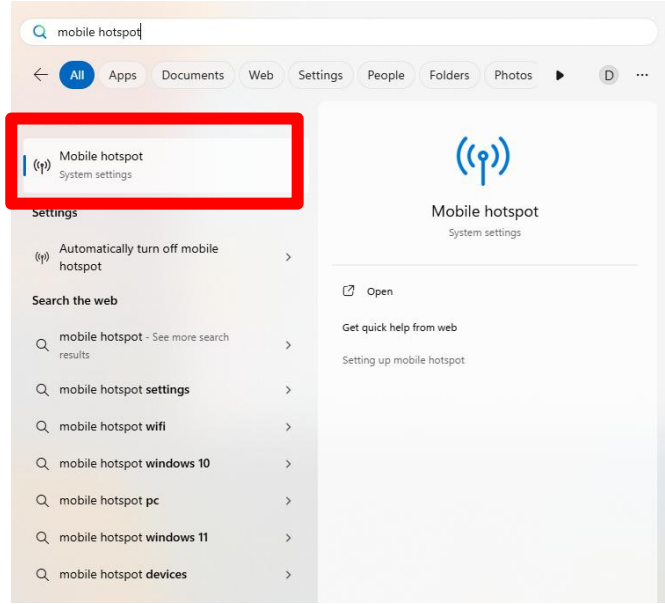
1. Connect Device to Host PC with USB-A to USB-C Cable (3)

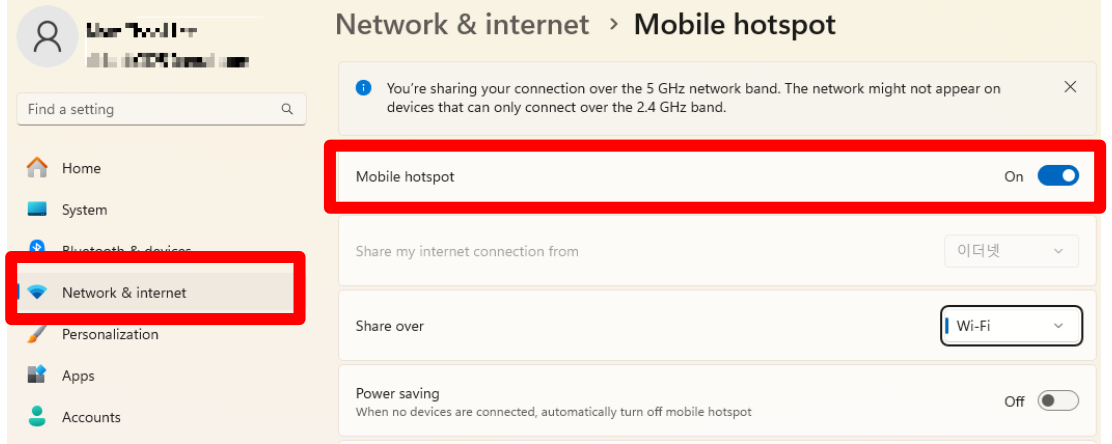
- Check the connection using ADB
  - It should show one device as below
    - The identifier of the device varies across devices

```
C:\Users\Lee>adb devices
List of devices attached
f10e86f9        device
```

2. Establish Wi-Fi Connection (1)

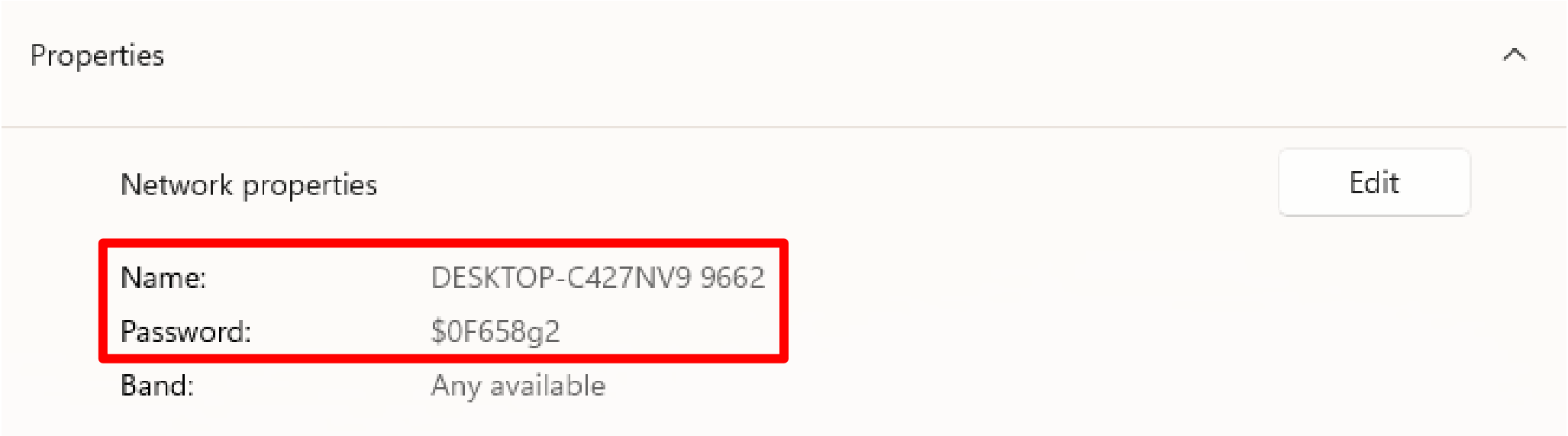
- Enable hotspot on host PC
  - Go to **Settings > Network & Internet**





2. Establish Wi-Fi Connection (2)

- Enable hotspot on host PC (cont'd)
  - Check the name and password of the activated network



2. Establish Wi-Fi Connection (3)

- Access the device in a command line tool with the following commands
  1. Get root access
     - **`adb root`**
  2. Enter adb shell
     - **`adb shell`**
  3. Switch to root user
     - **`su`**

```
C:\Users\Lee>adb root
adbd is already running as root

C:\Users\Lee>adb shell
# su
root@RUBIKPi:/#
```

2. Establish Wi-Fi Connection (4)

- Connect RUBIK Pi to the mobile hotspot
  1. Check the available Wi-Fis using network manager CLI (**`nmcli`**) of Linux
     - **`nmcli device wifi list`**

```
root@RUBIKPi:~# nmcli device wifi list
IN-USE  BSSID              SSID                                MODE   CHAN  RATE >
                                                               Infra  2     270 M>
                                                               Infra  10    270 M>
                                                               Infra  149   270 M>
                                                               Infra  40    270 M>
                                                               Infra  1     270 M>
        16:18:C3:3F:17:9A  DESKTOP-C427NV9 9662                 Infra  149   270 M>
                                                               Infra  149   270 M>
                                                               Infra  1     270 M>
```

2. Establish Wi-Fi Connection (5)

- Connect RUBIK Pi to the mobile hotspot (cont'd)
  2. Connect to the mobile hotspot using nmcli
     - **`nmcli device wifi connect "SSID" –ask`**

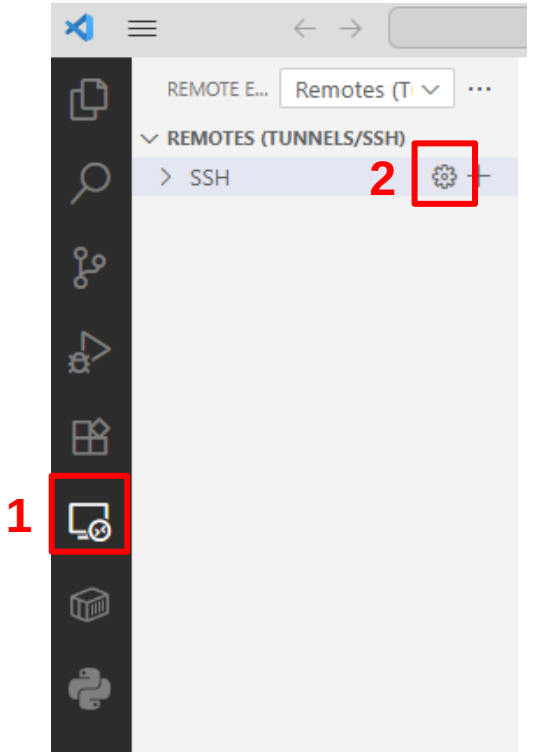
```
root@RUBIKPi:~# nmcli device wifi connect "DESKTOP-C427NV9 9662" --ask
Push of the WPS button on the router or a password is required to access the wireless
 network 'DESKTOP-C427NV9 9662'.
Password (802-11-wireless-security.psk): ********
Device 'wlan0' successfully activated with '0ea65cff-dd17-4aa1-b934-2dedbe33d58f'.
```

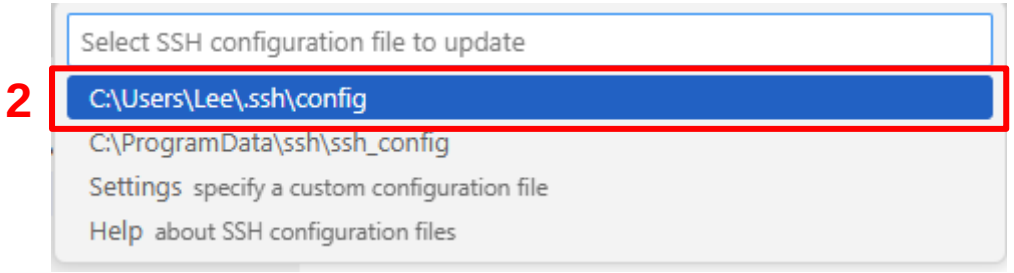
  3. Check the connection
     - **`ifconfig wlan0`**

```
root@RUBIKPi:~# ifconfig wlan0
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.137.35  netmask 255.255.255.0  broadcast 192.168.137.255
        inet6 fe80::72c0:c63d:66e2:c284  prefixlen 64  scopeid 0x20<link>
        ether 9c:b8:b4:9f:4f:5c  txqueuelen 1000  (Ethernet)
        RX packets 428199  bytes 627300827 (598.2 MiB)
        RX errors 0  dropped 17738  overruns 0  frame 0
        TX packets 89787  bytes 11357654 (10.8 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

3. Make SSH Connection (1)

- Create SSH configuration in VS Code
  1. Open remote explorer in VS Code in the left side bar
  2. Open SSH config file



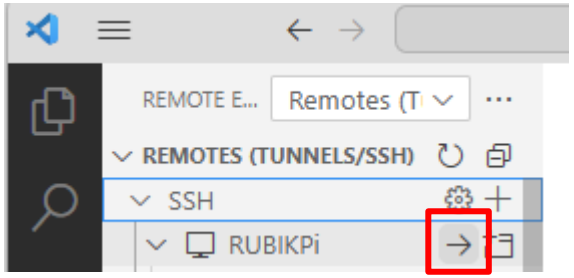


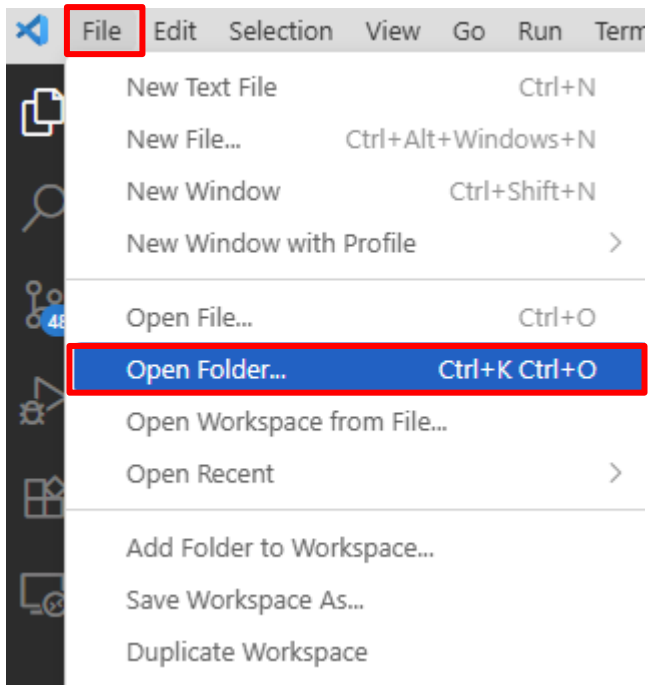
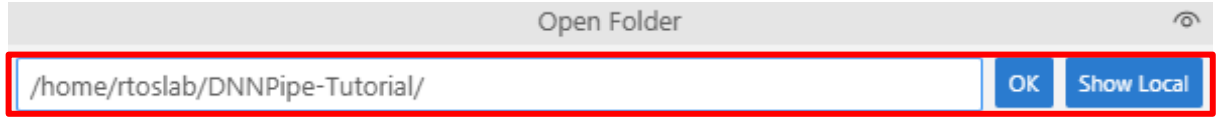
3. Make SSH Connection (2)

- Create SSH configuration in VS Code (cont'd)
  3. Add the configuration for RUBIK Pi

```
config
C: > Users > Lee > .ssh > config
1   Host RUBIKPi
2     HostName 192.168.137.35
3     Port 22
4     User rtoslab
```

  4. Connect to the RUBIK Pi with both user name and password being "**`rtoslab`**"



3. Make SSH Connection (3)

- Open **`~/DNNPipe-Tutorial`** directory